



ARCSOFT ARC FACE Developer's Guide

ARCSOFT ARC FACE

DEVELOPER'S GUIDE



©2025 ArcSoft Corporation Limited. All rights reserved.

版本日志

版本	日期	更新说明
2.2.16524030204.1	01/15/2020	1. 支持包名激活
3.0.16524030204.1	03/02/2020	1. 支持活体检测
3.0.16524030204.2	07/28/2020	1. 支持iOS14
3.0.16524030204.3	01/20/2021	1. 人脸比对优化
3.0.16524030204.4	08/25/2021	1. 支持iOS15
3.0.16524030204.5	08/02/2022	1. 支持iOS16
3.0.16524030204.6	09/08/2022	1. 修复偶现识别卡顿问题
3.1.16524030204.1	06/16/2023	1. 优化活体检测效果 2. 支持iOS17
3.1.16524030204.2	07/18/2024	1. 支持iOS18
3.1.16524030204.3	06/12/2025	1. 支持iOS26
5.0.16524030204.6	11/14/2025	1.增加人脸特征相关操作 2.增加交互式和炫彩活体相关操作
5.0.16524030204.14	12/02/2025	1. 兼容低版本设备
5.0.16524030204.18	12/31/2025	1、扩展特征长度为2056字节，向下兼容1032字节长度特征

1. 简介

1.1 产品概述

1.2 产品功能简介

1.2.1 人脸检测

1.2.2 人脸追踪

1.2.3 人脸特征提取

1.2.4 人脸特征比对

1.2.5 人脸属性检测

1.3 授权说明

1.4 环境要求

1.4.1 运行环境

1.4.2 系统要求

1.4.3 权限申明

2. 接入须知

2.1 SDK包结构

2.2 业务流程

2.2.1 通用流程概述

2.2.2 人脸 1:1 比对

2.2.2.1 图片vs图片

2.2.2.2 实时视频流vs图片

2.2.3 人脸 1:N 搜索

2.2.4 方案选型

2.3 指标

2.3.1 算法性能

2.3.2 阈值推荐

2.3.3 图像要求

3. SDK的详细接入介绍

3.1 Demo工程配置

3.2 支持的颜色格式

3.3 枚举类型

3.3.1 ASF_DetectMode

3.3.2 ASF_OrientPriority

3.3.3 ASF_DetectModel

3.3.4 ASF_ColorCode

3.3.5 ASF_FlashColor

3.3.6 ASF_ActionType

3.4 数据结构

3.4.1 ArcSoftActiveInfo

3.4.2 ASF_VERSION

3.4.3 ASF_FaceDataInfo

3.4.4 ASF_SingleFaceInfo

3.4.5 ASF_MultiFaceInfo

3.4.6 ASF_FaceFeature

3.4.7 ASF_AgeInfo

3.4.8 ASF_GenderInfo

- 3.4.9 ASF_Face3DAngleInfo
- 3.4.10 ASF_LivenessThreshold
- 3.4.11 ASF_LivenessInfo
- 3.4.12 ASF_FaceFeatureInfo
- 3.4.13 ASVLOFFSCREEN

3.5 功能接口

- 3.5.1 active
- 3.5.2 getActiveFileInfo
- 3.5.3 initFaceEngine
- 3.5.4 detectFaces
- 3.5.5 extractFaceFeature
- 3.5.6 compareFace
- 3.5.7 searchFaceFeature
- 3.5.8 searchFaceFeatureTopN
- 3.5.9 registerFaceFeature
- 3.5.10 registerSingleFaceFeature
- 3.5.11 removeFaceFeature
- 3.5.12 clearAllFaceFeature
- 3.5.13 updateFaceFeature
- 3.5.14 getFaceFeature
- 3.5.15 getFaceCount
- 3.5.16 detectLivenessInteractive
- 3.5.17 detectLivenessGlare
- 3.5.18 process
- 3.5.19 getAge
- 3.5.20 getGender
- 3.5.21 getVersion
- 3.5.22 setLivenessThreshold
- 3.5.23 getLivenessThreshold
- 3.5.24 getLiveness
- 3.5.25 unInitFaceEngine

4. 常见问题

- 4.1 常用接入场景流程
 - 4.1.1 常用比对流程
- 4.2 错误码列表
- 4.3 FAQ

1. 简介

1.1 产品概述

ArcFace 离线SDK，包含人脸检测、性别检测、年龄检测、人脸识别等能力，初次使用时需联网激活，激活后即可在本地无网络环境下工作，可根据具体的业务需求结合人脸识别SDK灵活地进行应用层开发。



1.2 产品功能简介

1.2.1 人脸检测

对传入的图像数据进行人脸检测，返回人脸的边框以及朝向信息，可用于后续的人脸识别、特征提取等操作；

- 支持IMAGE模式和VIDEO模式人脸检测。
- 支持单人脸、多人脸检测，最多支持检测人脸数为10。

1.2.2 人脸追踪

对来自于视频流中的图像数据，进行人脸检测，并对检测到的人脸进行持续跟踪。

1.2.3 人脸特征提取

提取人脸特征信息，用于人脸的特征比对。

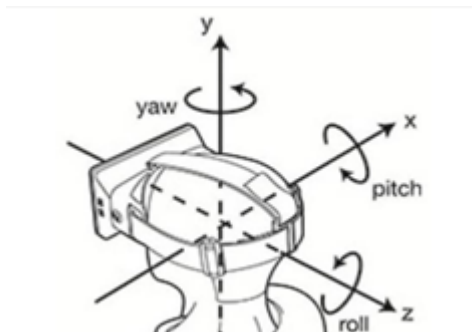
1.2.4 人脸特征比对

对两个人脸特征数据进行比对，返回比对相似度值。

1.2.5 人脸属性检测

人脸属性，支持检测年龄、性别、活体以及3D角度。

人脸3D角度：俯仰角（pitch），横滚角（roll），偏航角（yaw）。



1.3 授权说明

SDK授权按设备进行授权，每台硬件设备需要一个独立的授权，此授权的校验是基于设备的唯一标识，被授权的设备在初次授权时需在线进行授权，授权成功后可以离线运行SDK。

在线授权：

1. 确保可以正常访问公网；
2. 调用在线激活接口激活SDK；

注意事项：

1. 设备授权后，若设备授权信息被删除（重装系统/应用被卸载等），需联网重新激活；
2. 硬件信息发生变更，需要重新激活；

1.4 环境要求

1.4.1 运行环境

- iOS armv64

1.4.2 系统要求

- 支持iOS 9及以上

1.4.3 权限申明

- 允许应用联网，用于SDK联网激活授权

2. 接入须知

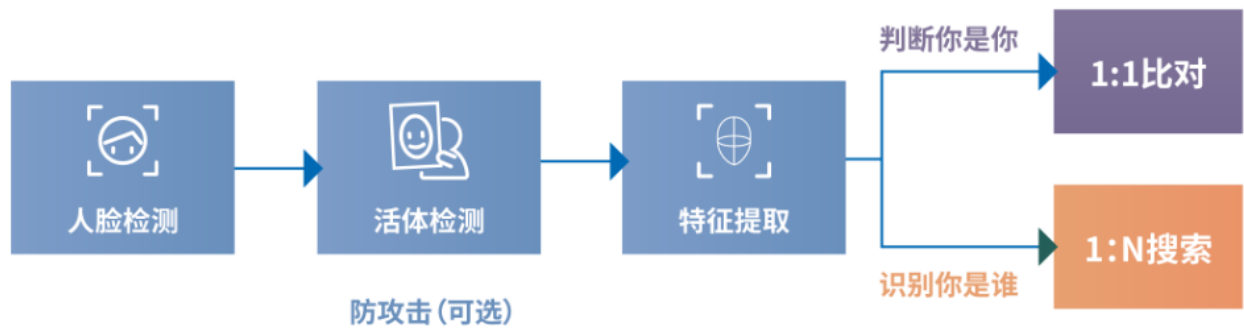
2.1 SDK包结构

---doc	
---ARCSOFT_ARC_FACE_DEVELOPER'S_GUIDE.pdf	开发说明文档
---libs	
---ArcSoftFaceEngine.framework	引擎库
---samplecode	
---ArcSoftFaceEngineDemo	示例工程
---releasenotes.txt	说明文件

2.2 业务流程

2.2.1 通用流程概述

根据实际应用场景以及硬件配置，活体检测和特征提取可选择是否采用并行的方式。



- 如上图所示，人脸识别的核心业务流程可以分为以下三个步骤：
 1. 人脸检测：采集视频帧或静态图，传入算法进行检测，输出人脸数据，用于后续的检测。
 2. 特征提取：对待比对的图像进行特征提取、人脸库的预提取，用于之后的比对；
 3. 人脸识别：1：1比对主要是判断「你是你」，用于核实身份的真实性；1：N搜索主要识别「你是谁」，用于明确身份的具体所属。

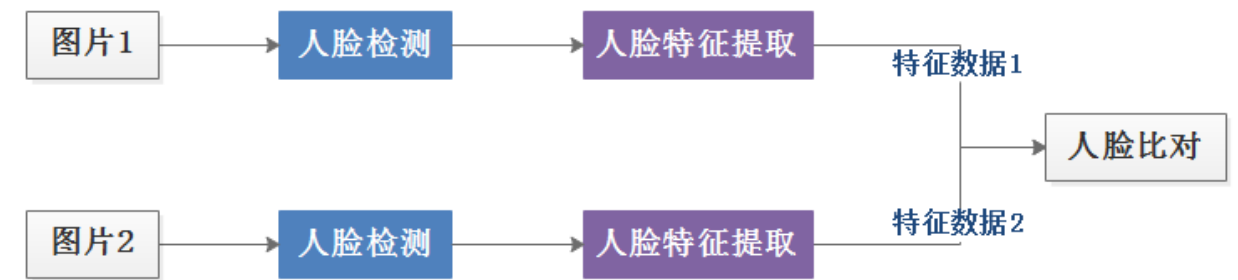
2.2.2 人脸 1:1 比对

1:1 比对常见的应用场景可分为如下两种形式：

2.2.2.1 图片vs图片

两张静态图片分别提取特征，进行比对。

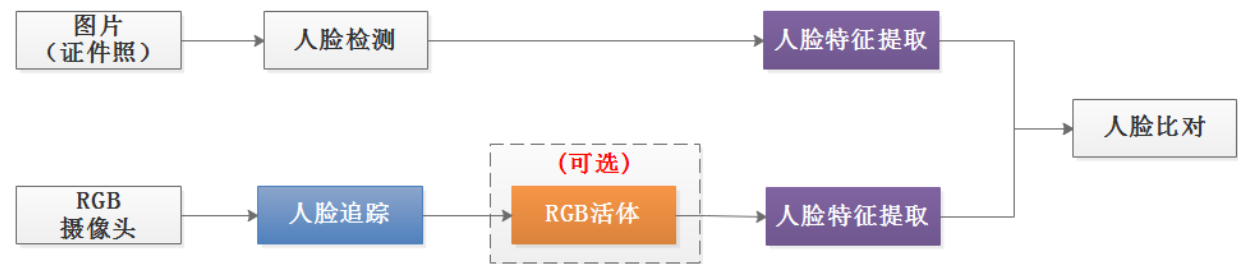
图片VS图片



2.2.2.2 实时视频流vs图片

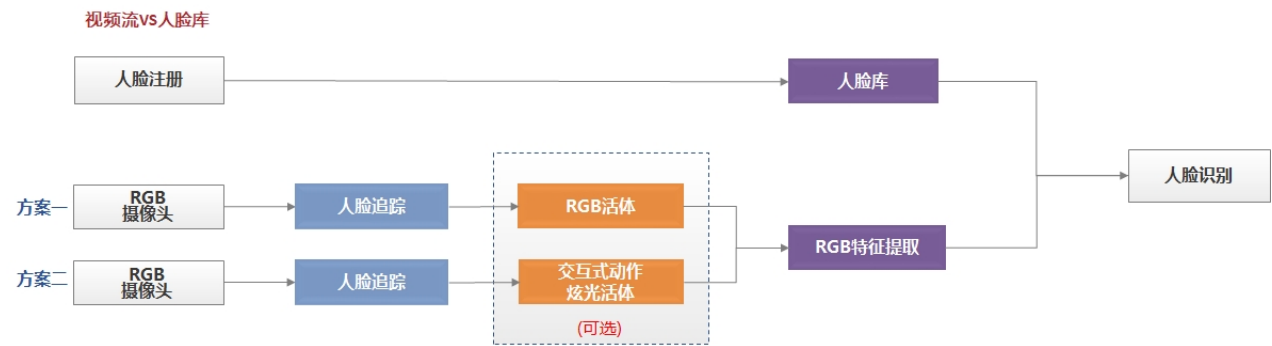
该场景比较常见，典型的应用场景为人证比对和人脸门禁，本SDK同时支持人证比对和生活照识别，在特征比对时选择对应的特征比对模型即可。

视频流VS图片



2.2.3 人脸 1:N 搜索

将需要识别的人脸图片集注册到本地人脸库中，当有用户需要识别身份时，从视频流中实时采集人脸图片，与人脸库中的人脸集合对比，得到搜索结果。



2.2.4 方案选型

可结合应用场景进行方案的选择

离线1：1比对

提供本地化的1：1人脸对比功能，证明你是你，可有效应用于车站、机场、大型活动、机关单位、银行、酒店、网吧等人员流动频繁场所或其它重点场景的人证核验，也可用于线上开户的人员身份验证等场景。

离线1：N检索

提供本地化的1：N人脸搜索功能，识别你是谁，可有效应用于社区、楼宇、工地、学校等较大规模的人脸考勤签到、人脸通行等应用场景。该版本人脸库在1万以内效果更佳。

2.3 指标

算法检测指标受硬件配置、图像数据质量、测试方法等因素影响，以下实验数据仅供参考，具体数据以实际应用场景测试结果为准。

2.3.1 算法性能

- 硬件信息：
 - iPhone 14Pro

算法	性能(ms)
Detect	< 30
FeatureExtract	< 10
FeatureCompare	< 0.009

2.3.2 阈值推荐

人脸比对分值区间为[0~1]，推荐阈值如下，高于此阈值的即可判断为同一人。

- 用于生活照之间的特征比对，推荐阈值0.80
- 用于证件照特征比对，推荐阈值0.80

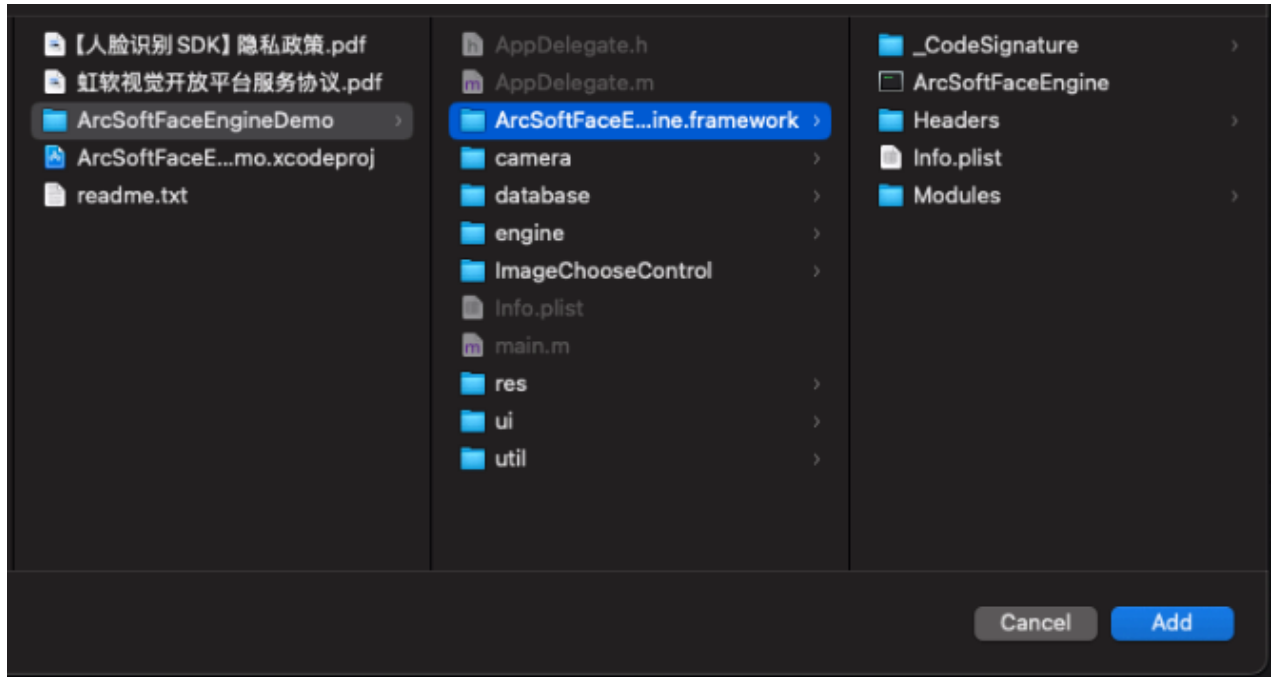
2.3.3 图像要求

- 建议待检测的图像人脸角度上、下、左、右转向小于30度；
- 图像中人脸尺寸不小于50 x 50像素；
- 图像大小小于10MB；
- 图像清晰；

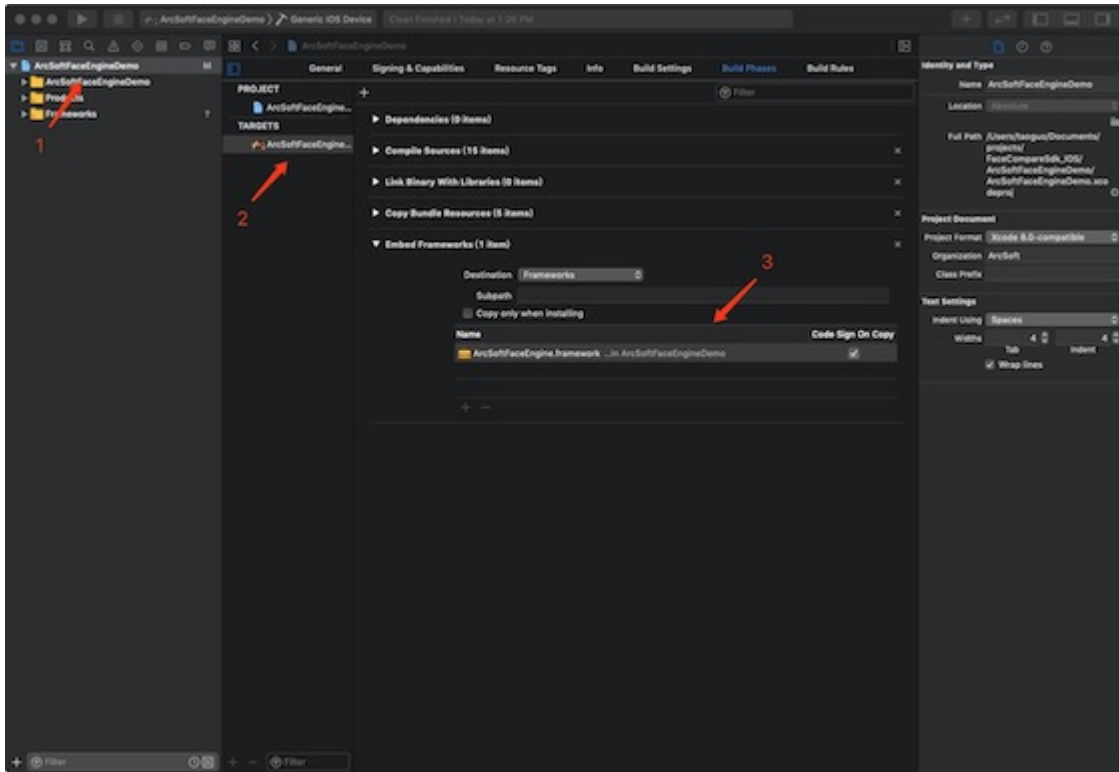
3. SDK的详细接入介绍

3.1 Demo工程配置

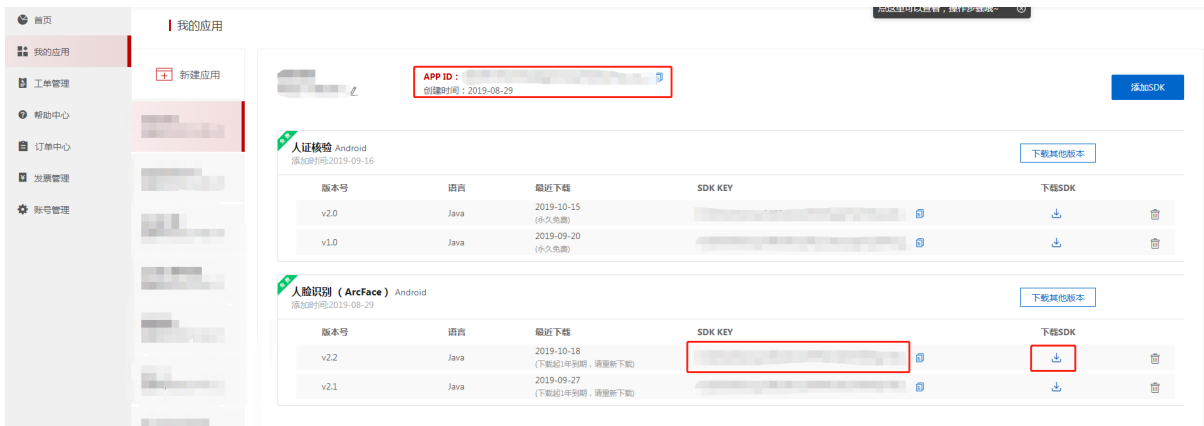
1. 打开SDK包中，samplecode 目录下的 ArcSoftFaceEngineDemo 工程
2. 把libs目录下的ArcSoftFaceEngine.framework复制到工程路径中



3. 确保工程如下图配置



4. 前往[虹软视觉开放平台](#)获取 APPID 、 SDKKEY



5. 将获取到的 APPID 、 SDKKEY 填入工程的指定位置

将 APPID 与 SDKKEY 赋值给激活函数 engineActive 对应的变量

6. 一切准备就绪，点击运行即可

3.2 支持的颜色格式

常量名	常量值	颜色格式说明
ASVL_PAF_NV12	2049	8-bit Y 通道，8-bit 2x2 采样 U 与 V 分量交织通道
ASVL_PAF_RGB24_B8G8R8	513	RGB 分量交织，按 B, G, R, B 字节序排布
ASVL_PAF_NV21	2050	8-bit Y 通道，8-bit 2x2 采样 V 与 U 分量交织通道

3.3 枚举类型

3.3.1 ASF_DetectMode

枚举描述

检测模式

枚举值描述

枚举名	描述
ASF_DETECT_MODE_VIDEO	VIDEO检测模式，用于处理连续帧的图像数据
ASF_DETECT_MODE_IMAGE	IMAGE检测模式，用于处理单张的图像数据

3.3.2 ASF_OrientPriority

枚举描述

检测角度优先级

枚举值描述

枚举名	描述
ASF_OP_0_ONLY	人脸检测角度，逆时针0度
ASF_OP_90_ONLY	人脸检测角度，逆时针90度
ASF_OP_180_ONLY	人脸检测角度，逆时针180度
ASF_OP_270_ONLY	人脸检测角度，逆时针270度
ASF_OP_ALL_OUT	人脸检测角度，全角度检测

3.3.3 ASF_DetectModel

枚举描述

检测模型

枚举值描述

枚举名	描述
ASF_DETECT_MODEL_RGB	RGB图像检测模型

3.3.4 ASF_ColorCode

枚举描述

炫光颜色顺序

枚举值描述

枚举名	描述
ASF_WRGB	白红绿蓝
ASF_WRGP	白红绿粉
ASF_WRGY	白红绿黄

3.3.5 ASF_FlashColor

枚举描述

炫光颜色

枚举值描述

枚举名	描述
ASF_COLOR_WHITE	白色
ASF_COLOR_RED	红色
ASF_COLOR_GREEN	绿色
ASF_COLOR_BLUE	蓝色
ASF_COLOR_PINK	粉色

枚举名	描述
ASF_COLOR_YELLOW	黄色
ASF_COLOR_BLACK	黑色

3.3.6 ASF_ActionType

枚举描述

交互式动作

枚举值描述

枚举名	描述
ASF_ACTION_TYPE_U	未知
ASF_ACTION_TYPE_EB	眨眼
ASF_ACTION_TYPE_MO	张嘴
ASF_ACTION_TYPE_HL	左摆头
ASF_ACTION_TYPE_HR	右摆头

3.4 数据结构

3.4.1 ArcSoftActiveInfo

类描述

激活文件信息。

成员描述

类型	变量名	描述
NSString	appld	APP ID
NSString	sdkKey	SDK KEY
NSString	platform	平台版本
NSString	sdkType	SDK类型
NSString	sdkVersion	SDK版本号

类型	变量名	描述
NSString	fileVersion	激活文件版本号
NSString	startTime	SDK开始时间
NSString	endTime	SDK截止时间

3.4.2 ASF_VERSION

结构体描述

SDK版本信息。

成员描述

```
typedef char* MPChar;
```

类型	变量名	描述
MPChar	Version	版本号
MPChar	BuildDate	构建日期
MPChar	CopyRight	版权说明

3.4.3 ASF_FaceDataInfo

结构体描述

人脸信息。

成员描述

```
typedef void* MPVoid;  
typedef signed int MInt32;
```

类型	变量名	描述
MPVoid	data	人脸信息
MInt32	dataSize	人脸信息长度

3.4.4 ASF_SingleFaceInfo

结构体描述

单人脸信息。

成员描述

typedef signed int MInt32;

```
typedef struct __tag_rect
{
    MInt32 left;
    MInt32 top;
    MInt32 right;
    MInt32 bottom;
} MRECT, *PMRECT;
```

类型	变量名	描述
MRECT	faceRect	人脸框
MInt32	faceOrient	人脸角度
ASF_FaceDataInfo	faceDataInfo	单张人脸数据

3.4.5 ASF_MultiFaceInfo

结构体描述

多人脸信息。

成员描述

typedef signed int MInt32;

```
typedef struct __tag_rect
{
    MInt32 left;
    MInt32 top;
    MInt32 right;
    MInt32 bottom;
} MRECT, *PMRECT;
```

类型	变量名	描述

类型	变量名	描述
MInt32	faceNum	检测到的人脸数
MRECT*	faceRect	人脸框数组
MInt32*	faceOrient	人脸角度数组
MInt32*	faceID	一张人脸从进入画面直到离开画面，faceID不变。在VIDEO模式下有效，IMAGE模式下为空。
LPASF_FaceDataInfo	faceDataInfoList	多张人脸信息
MInt32*	facelsWithinBoundary	人脸是否在边界内 0 人脸溢出；1 人脸在图像边界内
MRECT*	foreheadRect	人脸额头区域
ASF_Face3DAngleInfo	face3DAngleInfo	人脸3D角度

3.4.6 ASF_FaceFeature

结构体描述

人脸特征。

成员描述

```
typedef signed int MInt32;

typedef unsigned char MByte;
```

类型	变量名	描述
MByte*	feature	人脸特征
MInt32	featureSize	人脸特征长度

3.4.7 ASF_AgeInfo

结构体描述

年龄信息。

成员描述

```
typedef signed int MInt32;
```

类型	变量名	描述
MInt32*	ageArray	0:未知; >0:年龄
MInt32	num	检测的人脸数

3.4.8 ASF_GenderInfo

结构体描述

性别信息。

成员描述

```
typedef signed int MInt32;
```

类型	变量名	描述
MInt32*	genderArray	0:男性; 1:女性; -1:未知
MInt32	num	检测的人脸数

3.4.9 ASF_Face3DAngleInfo

结构体描述

3D角度信息。

成员描述

```
typedef signed int MInt32;
```

```
typedef float MFloat;
```

类型	变量名	描述
MFloat*	roll	横滚角
MFloat*	yaw	偏航角
MFloat*	pitch	俯仰角

3.4.10 ASF_LivenessThreshold

结构体描述

活体置信度。

成员描述

```
typedef float MFloat;
```

类型	变量名	描述
MFloat	thresholdmodel_BGR	BGR活体检测阈值设置，默认值0.5

3.4.11 ASF_LivenessInfo

结构体描述

活体信息。

成员描述

```
typedef signed int MInt32;
```

类型	变量名	描述
MInt32*	isLive	0: 非真人; 1: 真人; -1: 不确定; -2: 传入人脸数 > 1; -3: 人脸过小; -4: 角度过大; -5: 人脸超出边界;
MInt32	num	检测的人脸个数

3.4.12 ASF_FaceFeatureInfo

结构体描述

搜索人脸信息。

成员描述

```
typedef signed int MInt32;
```

类型	变量名	描述
MInt32	searchId	唯一标识符
LPASF_FaceFeature	feature	人脸特征值

类型	变量名	描述
MPCChar	tag	备注

3.4.13 ASVLOFFSCREEN

结构体描述

图像数据信息，该结构体在asvloffscreen.h基础的头文件中。

成员描述

```
typedef LPASVLOFFSCREEN LPASF_ImageData;

typedef unsigned int MUInt32;

typedef signed int MInt32;

typedef unsigned char MUInt8;
```

类型	变量名	描述
MUInt32	u32PixelFormat	颜色格式
MInt32	i32Width	图像宽度
MInt32	i32Height	图像高度
MUInt8**	ppu8Plane	图像数据
MInt32*	pi32Pitch	图像步长

3.5 功能接口

当前版本使用同一个引擎句柄不支持多线程调用同一个算法接口，若需要对同一个接口进行多线程调用需要启动多个引擎。

例如：若需要做多人脸门禁系统，我们希望提升识别速度，一般会使用同一个引擎进行人脸检测，我们可以提前初始化多个引擎用于人脸特征提取，这些引擎只需要初始化 ASF_FACERECOGNITION 属性即可。同时要根据硬件配置来控制最多启动的线程数。

3.5.1 active

功能描述

用于在线激活SDK。

方法

```
- (MRESULT)activewithAppId:(NSString *)appId SDKKey:(NSString *)sdkkey;
```

初次使用SDK时需要对SDK先进行激活，激活后无需重复调用；
调用此接口时必须为联网状态，激活成功后即可离线使用；

参数说明

参数	类型	描述
appId	in	官网获取的APPID
sdkKey	in	官网获取的SDKKEY

返回值

成功返回MOK、MERR_ASF_ALREADY_ACTIVATED，失败详见 [4.2 错误码列表](#)。

示例代码

```
//该组数据无效，仅做示例参考
NSString* appId = @"D617np8jyKt1jN9gMr7ENbT3gjFdaeDet3Pp8yfwHxnt"
NSString* sdkKey = @"8FdAtZSffuvS6kuzPP4PrxyxkQMGPsi4yE3Amo61sbF2"

ArcSoftFaceEngine *engine = [ArcSoftFaceEngine alloc]init];
[engine activewithAppId:appId SDKKey:sdkKey];
```

3.5.2 getActiveFileInfo

功能描述

获取激活文件信息。

方法

```
- (MRESULT)getActiveFileInfo:(ArcSoftActiveInfo*)activeInfo;
```

参数说明

参数	类型	描述
activeFileInfo	out	激活文件信息

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
ArcSoftActiveInfo* info = [ArcSoftActiveInfo new];
[engine getActiveFileInfo:info];
```

3.5.3 initFaceEngine

功能描述

初始化引擎。

该接口至关重要，清楚的了解该接口参数的意义，可以避免一些问题以及对项目的设计都有一定的帮助。

方法

```
- (MRESULT)initFaceEngineWithDetectMode:(ASF_DetectMode)detectMode
                                orientPriority:(ASF_OrientPriority)detectFaceOrientPriority
                                maxFaceNum:(MInt32)detectFaceMaxNum
                                combinedMask:(MInt32)combinedMask;
```

参数说明

参数	类型	描述
detectMode	in	VIDEO模式：处理连续帧的图像数据 IMAGE模式：处理单张的图像数据
detectFaceOrientPriority	in	人脸检测角度，推荐单一角度检测；
detectFaceMaxNum	in	最大需要检测的人脸个数，取值范围[1,10]
combinedMask	in	需要启用的功能组合，可多选

重点参数详细说明

• **detectMode**

枚举名	说明
ASF_DETECT_MODE_VIDEO	VIDEO模式
ASF_DETECT_MODE_IMAGE	IMAGE模式

VIDEO 模式

- 1. 对视频流中的人脸进行追踪，人脸框平滑过渡，不会出现跳框的现象。
- 2. 用于预览帧数据的人脸追踪，使用该模式比使用IMAGE模式处理速度更快，可避免出现卡顿问题。

IMAGE 模式

- 1. 针对单张图片进行人脸检测精度更高。
- 2. 在注册人脸库时，我们建议使用精度更高的IMAGE模式。

模式选择

- 1. 从摄像头中获取数据并需要预览显示，推荐选择VIDEO模式；
- 2. 处理静态图像数据，类似注册人脸库时，推荐使用IMAGE模式；
- 3. 若同时需要进行IMAGE模式人脸检测和VIDEO模式人脸检测，需要创建一个VIDEO模式的引擎和一个IMAGE模式的引擎；

• **detectFaceOrientPriority**

枚举名	说明
ASF_OP_0_ONLY	人脸检测角度，逆时针0度
ASF_OP_90_ONLY	人脸检测角度，逆时针90度
ASF_OP_180_ONLY	人脸检测角度，逆时针180度
ASF_OP_270_ONLY	人脸检测角度，逆时针270度
ASF_OP_ALL_OUT	人脸检测角度，全角度检测

注：设置0度方向并不是说只有0度角可以检测到人脸，而是大体在这个方向上的人脸均可以，我们也提供了全角度方向检测，但在应用场景确定的情况下我们还是推荐使用单一角度进行检测，因为单一角相对于全角度性能更好、精度更高。



0度角



90度角



180度角



270度角

• combinedMask

针对算法功能会有常量值与之一一对应，可根据业务需求进行自由选择，不需要的属性可以不用初始化，否则会占用多余内存。

```
#define ASF_FACE_DETECT          0x00000001    //此处detect可以是tracking或者
detection两个引擎之一，具体的选择由detect mode 确定
#define ASF_FACERECOGNITION      0x00000004    //人脸特征
#define ASF_AGE                  0x00000008    //年龄
#define ASF_GENDER               0x00000010    //性别
#define ASF_LIVENESS             0x00000080    //RGB活体
#define ASF_LIVENESS_SCREENFLASH 0x00002000    //交互式炫光活体
```

说明：

1. 人脸识别时一般都需要 ASF_FACE_DETECT 和 ASF_FACERECOGNITION 这两个属性。
2. ASF_AGE / ASF_GENDER / ASF_LIVENESS / ASF_LIVENESS_SCREENFLASH 根据业务需求进行选择即可。

这些属性均是以常量值进行定义，可通过 `|` 位运算符进行组合使用。

例如：int combinedMask = FaceEngine.ASF_FACE_DETECT | FaceEngine.ASF_FACERECOGNITION ;

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
MInt32 combineMask = ASF_FACE_DETECT | ASF_FACERECOGNITION;
engine = [[ArcSoftFaceEngine alloc] init];
[engine initWithFaceEngineWithDetectMode:ASF_DETECT_MODE_IMAGE
orientPriority:ASF_OP_ALL_OUT maxFaceNum:1 combinedMask:combineMask];
```

3.5.4 detectFaces

功能描述

检测人脸信息。

该功能依赖初始化的模式选择，VIDEO模式下使用的是人脸追踪功能，IMAGE模式下使用的是人脸检测功能。初始化中detectFaceOrientPriority、detectFaceScaleVal、detectFaceMaxNum参数的设置，对能否检测到人脸以及检测到几张人脸都有决定性的作用。

方法

```
- (MRESULT)detectFacesWithWidth:(MInt32)width
                        height:(MInt32)height
                        data:(MUInt8*)data
                        format:(MInt32)format
                        faceRes:(LPASF_MultiFaceInfo)detectedFaces;
```

参数说明

参数	类型	描述
data	in	图像数据
width	in	图像宽度，为4的倍数
height	in	图像高度，在NV12、NV21格式下要求为2的倍数 BGR24格式无限制
format	in	支持NV12/NV21/BGR24
detectedFaces	out	检测到的人脸信息

返回值

成功返回MOK，失败详见[4.2 错误码列表](#)。

detectFaceMaxNum 参数的设置，对能否检测到人脸以及检测到几张人脸都有决定性的作用。

示例代码

```
ASF_MultiFaceInfo faceInfo = { 0 };
[engine detectFacesWithWidth:picwidth height:picHeight data:nv12Data format:format
faceRes:&faceInfo];
```

3.5.5 extractFaceFeature

功能描述

单人脸特征信息提取。

方法

```
- (MRESULT)extractFaceFeatureWithwidth:(MInt32)width
                        height:(MInt32)height
                        data:(MUInt8*)data
                        format:(MInt32)format
                        faceInfo:(LPASF_SingleFaceInfo)faceInfo
                        feature:(LPASF_FaceFeature)feature;
```

参数说明

参数	类型	描述
data	in	图像数据
width	in	图片宽度，为4的倍数
height	in	图像高度，在NV12、NV21格式下要求为2的倍数 BGR24格式无限制
format	in	支持NV12/NV21/BGR24
faceInfo	in	人脸信息（人脸框、人脸角度）
feature	out	提取到的人脸特征信息

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

依赖 detectFaces 接口成功检测到人脸，将检测到的人脸信息取单张人脸信息和使用的图像信息传入到该接口进行特征提取。

```
ASF_FaceFeature feature = {0};
ASF_SingleFaceInfo frInputFace = {0};
frInputFace.rcFace.left = fdResult->faceRect[0].left;
frInputFace.rcFace.top = fdResult->faceRect[0].top;
frInputFace.rcFace.right = fdResult->faceRect[0].right;
frInputFace.rcFace.bottom = fdResult->faceRect[0].bottom;
frInputFace.orient = fdResult->faceOrient[0];
frInputFace.faceDataInfo = fdResult->faceDataInfoList[0];
[engine extractFaceFeatureWithWidth:picwidth height:picHeight data:nv12Data
format:format faceInfo:&frInputFace feature:&feature];
```

3.5.6 compareFace

功能描述

人脸特征比对，输出相似度。

方法

- (MRESULT)compareFaceWithFeature:(LPASF_FaceFeature)feature1
feature2:(LPASF_FaceFeature)feature2
confidenceLevel:(MFloat*)confidenceLevel;
- (MRESULT)compareFaceWithFeature:(LPASF_FaceFeature)feature1
feature2:(LPASF_FaceFeature)feature2
compareModel:(ASF_CompareModel)compareModel
confidenceLevel:(MFloat*)confidenceLevel;

参数说明

参数	类型	描述
feature1	in	人脸特征
feature2	in	人脸特征
compareModel	in	可选，用于设置当前比对的照片是注册照还是生活照，默认生活照
confidenceLevel	out	比对相似度

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
ASF_FaceFeature feature1 = {0};
ASF_FaceFeature feature2 = {0};
//获取需要对比的feature
MFloat confidence = 0;
[engine compareFaceWithFeature:feature1 feature2:&feature2
confidenceLevel:&confidence];
```

3.5.7 searchFaceFeature

功能描述

查询单个人脸特征，一般用于1VN比对

方法

```
- (MRESULT)searchFaceFeatureWithFeature:(LPASF_FaceFeature) feature
    compareModel:(ASF_CompareModel)compareModel
    similarity:(MFloat*)confidenceLevel
    faceFeatureInfo:(LPASF_FaceFeatureInfo) featureInfo;
```

参数说明

参数	类型	描述
feature	in	人脸特征
compareModel	in	比对模式
confidenceLevel	in	置信度
featureInfo	out	最高置信度对应的人脸特征信息

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```

ASF_FaceFeatureInfo searchResult;
mr = [self.arcsoftFace searchFaceFeatureWithFeature:&feature1
compareModel:ASF_LIFE_PHOTO similarity:&score faceFeatureInfo:&searchResult];
if (mr == MOK)
{
    NSLog(@"searchFaceFeatureWithFeature succeed,score %f, tag:%s", score,
searchResult.tag);
}
else
{
    NSLog(@"searchFaceFeatureWithFeature failed errcode %ld", mr);
}

```

3.5.8 searchFaceFeatureTopN

功能描述

查询topN个人脸特征

方法

```

- (MRESULT)searchFaceFeatureTopNWithFeature:(LPASF_FaceFeature) feature
searchNum:(MInt32)topN
compareModel:(ASF_CompareModel)compareModel
faceSearchResultGroup:
(LPASF_FaceFeatureSearchResult)faceSearchResultGroup;

```

参数说明

参数	类型	描述
feature	in	人脸特征
topN	in	需要查询的人脸特征数量
compareModel	in	比对模式
faceSearchResultGroup	out	查询的人脸特征结果

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
LPASF_FaceFeatureSearchResult pSearchRes = malloc (2 *
sizeof(ASF_FaceFeatureSearchResult));
mr = [self.arcsoftFace searchFaceFeatureTopNWithFeature:&feature1 searchNum:2
compareModel:ASF_LIFE_PHOTO faceSearchResultGroup:pSearchRes];
if (mr == MOK)
{
    NSLog(@"searchFaceFeatureWithFeature succeed,score %f, tag:%s",
pSearchRes[0].confidenceLevel, pSearchRes[0].stFaceFeatureInfo.tag);
}
else
{
    NSLog(@"searchFaceFeatureWithFeature failed errcode %ld", mr);
}
free(pSearchRes);
```

3.5.9 registerFaceFeature

功能描述

批量注册人脸特征信息

方法

```
- (MRESULT)registerFaceFeatureWithFeatureInfoList:(LPASF_FaceFeatureInfo)
featureInfoList
                                registerNum:(MInt32)size;
```

参数说明

参数	类型	描述
featureInfoList	in	人脸特征列表
size	in	需要查询的人脸特征数量

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```

ASF_FaceFeatureInfo faceFeatureList;
faceFeatureList.tag = "11345";
faceFeatureList.feature = &feature1;
faceFeatureList.searchId = 0;
mr = [self.arcsoftFace registerFaceFeatureWithFeatureInfoList:&faceFeatureList
registerNum:1];
if (mr == MOK)
{
    NSLog(@"registerFaceFeatureWithFeatureInfoList res %ld", mr);
} else {
    NSLog(@"registerFaceFeatureWithFeatureInfoList failed errcode %ld", mr);
}

```

3.5.10 registerSingleFaceFeature

功能描述

注册单个人脸特征信息

方法

```

- (MRESULT)registerSingleFaceFeatureWithFeatureInfo:(LPASF_FaceFeatureInfo)
featureInfo;

```

参数说明

参数	类型	描述
featureInfo	in	人脸特征

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码


```

ASF_FaceFeatureInfo faceFeatureInfo;
faceFeatureList.tag = "test";
faceFeatureList.feature = &feature1;
faceFeatureList.searchId = 0;
mr = [self.arcsoftFace registerSingleFaceFeatureWithFeatureInfo:&faceFeatureInfo];
if (mr == MOK)
{
    NSLog(@"registerSingleFaceFeatureWithFeatureInfo res %ld", mr);
} else {
    NSLog(@"registerSingleFaceFeatureWithFeatureInfo failed errcode %ld", mr);
}

```

3.5.11 removeFaceFeature

功能描述

删除单个人脸特征信息

方法

```
- (MRESULT) removeFaceFeatureWithSearchId: (MInt32) searchId;
```

参数说明

参数	类型	描述
searchId	in	人脸查询ID

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```

mr = [self.arcsoftFace removeFaceFeatureWithSearchId:0];

if (mr == MOK)
{
    NSLog(@"removeFaceFeatureWithSearchId succeed");
} else {
    NSLog(@"removeFaceFeatureWithSearchId failed", mr);
}

```

3.5.12 clearAllFaceFeature

功能描述

清空人脸特征信息

方法

```
- (MRESULT)clearAllFaceFeature;
```

参数说明

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
mr = [self.arcsoftFace clearAllFaceFeature];
```

3.5.13 updateFaceFeature

功能描述

更新已经注册的人脸特征信息

方法

```
- (MRESULT)updateFaceFeatureWithFeatureInfo:(LPASF_FaceFeatureInfo)featureInfo;
```

参数说明

参数	类型	描述
featureInfo	in	人脸特征信息

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```

faceFeatureList.tag = "78910";
mr = [self.arcsoftFace updateFaceFeatureWithFeatureInfo:&faceFeatureList];
if (mr == MOK)
{
    NSLog(@"updateFaceFeatureWithFeatureInfo res %ld", mr);
} else {
    NSLog(@"updateFaceFeatureWithFeatureInfo failed errcode %ld", mr);
}

```

3.5.14 getFaceFeature

功能描述

根据人脸ID获取人脸特征信息

方法

```

- (MRESULT)getFaceFeatureWithSearchId:(MInt32)searchId
    faceFeatureInfo:(LPASF_FaceFeatureInfo)featureInfo;

```

参数说明

参数	类型	描述
searchId	in	人脸ID
featureInfo	out	人脸特征信息

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

```

ASF_FaceFeatureInfo faceFeatureInfoToGet;
mr = [self.arcsoftFace getFaceFeatureWithSearchId:0
    faceFeatureInfo:&faceFeatureInfoToGet];
if (mr == MOK)
{
    NSLog(@"getFaceFeatureWithSearchId succeed");
} else {
    NSLog(@"getFaceFeatureWithSearchId failed errcode %ld", mr);
}

```

3.5.15 getFaceCount

功能描述

获取当前注册的人脸数量

方法

```
- (MRESULT)getFaceCount:(MInt32*)num;
```

参数说明

参数	类型	描述
num	in	人脸数量

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
MInt32 faceNum = 0;
mr = [self.arcsoftFace getFaceCount:&faceNum];
if (mr == MOK)
{
    NSLog(@"getFaceCount succeed,faceNum %d", faceNum);
} else {
    NSLog(@"getFaceCount failed errcode %ld", mr);
}
```

3.5.16 detectLivenessInteractive

功能描述

交互式活体检测

方法

```

- (MRESULT)detectLivenessInteractivewithwidth:(MInt32)width
    height:(MInt32)height
    format:(MInt32)format
    data:(MUInt8*)data
    singleFaceInfo:(LPASF_SingleFaceInfo)faceInfo
    action:(MInt32) iAction
    resetOpt:(MInt32) iReset
    livenessResult:(LPASF_LivenessResult)livenessResult;

```

参数说明

参数	类型	描述
width	in	图像宽度，为4的倍数
height	in	图像高度，在NV12、NV21格式下要求为2的倍数 BGR24格式无限制
format	in	支持NV12/NV21/BGR24
data	in	人脸图像数据
faceInfo	in	单张人脸信息
iAction	in	当前的交互式动作
iReset	in	复位（用于复位本次交互式动作流程）
livenessResult	out	交互式活体检测结果

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

```

uint8_t * imageDataTemp = uint8FromNSString(data);
//提取人脸特征
ASF_MultiFaceInfo stMulFaceInfoTemp;
ASF_SingleFaceInfo singleDetectedFacesTemp = { 0 };
mr = [self.arcsoftFace detectFaceswithwidth:width height:height data:imageDataTemp
format:ASVL_PAF_NV21 faceRes:&stMulFaceInfoTemp];
if (mr == MOK)
{
    NSLog(@"NV21 1 face detectface 成功");
    NSLog(@"facenum:%d ", stMulFaceInfoTemp.faceNum);
}

```

```

} else {
    NSLog(@"NV21 1 face detectface 失败: %ld", mr);
}
SingleDetectedFacesTemp.faceRect.left = stMulFaceInfoTemp.faceRect[0].left;
SingleDetectedFacesTemp.faceRect.top = stMulFaceInfoTemp.faceRect[0].top;
SingleDetectedFacesTemp.faceRect.right = stMulFaceInfoTemp.faceRect[0].right;
SingleDetectedFacesTemp.faceRect.bottom = stMulFaceInfoTemp.faceRect[0].bottom;
SingleDetectedFacesTemp.faceOrient = stMulFaceInfoTemp.faceOrient[0];
SingleDetectedFacesTemp.faceDataInfo = stMulFaceInfoTemp.faceDataInfoList[0];
ASF_LivenessResult liveRes;
mr = [self.arcsoftFace detectLivenessInteractivewithwidth:width height:height
format:ASVL_PAF_NV21 data:imageDataTemp singleFaceInfo:&SingleDetectedFacesTemp
action:ASF_ACTION_TYPE_EB resetOpt:0 livenessResult:&liveRes];
if (mr == MOK)
{
    NSLog(@"detectLivenessInteractivewithwidth succeed. num:%d live:%d", liveRes.iNum,
liveRes.iRes);
} else {
    NSLog(@"detectLivenessInteractivewithwidth failed errcode %ld", mr);
}

```

3.5.17 detectLivenessGlare

功能描述

炫光活体检测

方法

```

- (MRESULT)detectLivenessGlarewithwidth:(MInt32) width
                    height:(MInt32)height
                    format:(MInt32)format
                    data:(MUInt8*)data
singleFaceInfo:(LPASF_SingleFaceInfo)faceInfo
flashSequence:(MInt32) iMode
                    color:(MInt32) currentFlashColor
resetOpt:(MInt32) iReset
livenessResult:(LPASF_LivenessResult)livenessResult;

```

参数说明

参数	类型	描述
width	in	图像宽度，为4的倍数

参数	类型	描述
height	in	图像高度，在NV12、NV21格式下要求为2的倍数 BGR24格式无限制
format	in	支持NV12/NV21/BGR24
data	in	人脸图像数据
faceInfo	in	单张人脸信息
iMode	in	本次的炫光顺序模式
currentFlashColor	in	当前炫光的颜色
iReset	in	复位(用于复位本轮炫光检测)
livenessResult	out	交互式活体检测结果

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
//提取人脸特征
ASF_MultiFaceInfo stMulFaceInfoTemp;
ASF_SingleFaceInfo singleDetectedFacesTemp = { 0 };
mr = [self.arcsoftFace detectFacesWithWidth:width height:height data:imageDataTemp
format:ASVL_PAF_NV21 faceRes:&stMulFaceInfoTemp];
if (mr == MOK)
{
    NSLog(@"NV21 1 face detectface 成功");
    NSLog(@"facenum:%d ", stMulFaceInfoTemp.faceNum);
} else {
    NSLog(@"NV21 1 face detectface 失败: %ld", mr);
}
singleDetectedFacesTemp.faceRect.left = stMulFaceInfoTemp.faceRect[0].left;
singleDetectedFacesTemp.faceRect.top = stMulFaceInfoTemp.faceRect[0].top;
singleDetectedFacesTemp.faceRect.right = stMulFaceInfoTemp.faceRect[0].right;
singleDetectedFacesTemp.faceRect.bottom = stMulFaceInfoTemp.faceRect[0].bottom;
singleDetectedFacesTemp.faceOrient = stMulFaceInfoTemp.faceOrient[0];
singleDetectedFacesTemp.faceDataInfo = stMulFaceInfoTemp.faceDataInfoList[0];
ASF_LivenessResult liveRes;
```

```
mr = [self.arcsoftFace detectLivenessGlarewithwidth:width height:height format:
(ASVL_PAF_NV21) data:imageDataTemp singleFaceInfo:&SingleDetectedFacesTemp
flashSequence:ASF_WRGP color:color[i-1] resetOpt:0 livenessResult:&liveRes];

if (mr == MOK)
{
    NSLog(@"detectLivenessGlarewithwidth succeed. num:%d live:%d", liveRes.iNum,
liveRes.iRes);
} else {
    NSLog(@"detectLivenessGlarewithwidth failed errcode %ld", mr);
}
```

3.5.18 process

该接口仅支持可见光图像检测。

功能描述

人脸属性检测（年龄/性别/），最多支持4张人脸信息检测，超过部分返回未知。

方法

```
- (MRESULT)processwithwidth:(MInt32)width
                    height:(MInt32)height
                    data:(MUInt8*)data
                    format:(MInt32)format
                    faceRes:(LPASF_MultiFaceInfo)detectedFaces
                    mask:(MInt32)combinedMask;
```

参数说明

参数	类型	描述
data	in	图像数据
width	in	图片宽度，为4的倍数
height	in	图像高度，在NV12、NV21格式下要求为2的倍数 BGR24格式无限制
format	in	支持NV12/NV21/BGR24
faceInfoList	in	人脸信息列表

参数	类型	描述
combinedMask	in	检测的属性（ASF_AGE、ASF_GENDER、ASF_LIVENESS），支持多选 检测的属性须在引擎初始化接口的combinedMask参数中启用

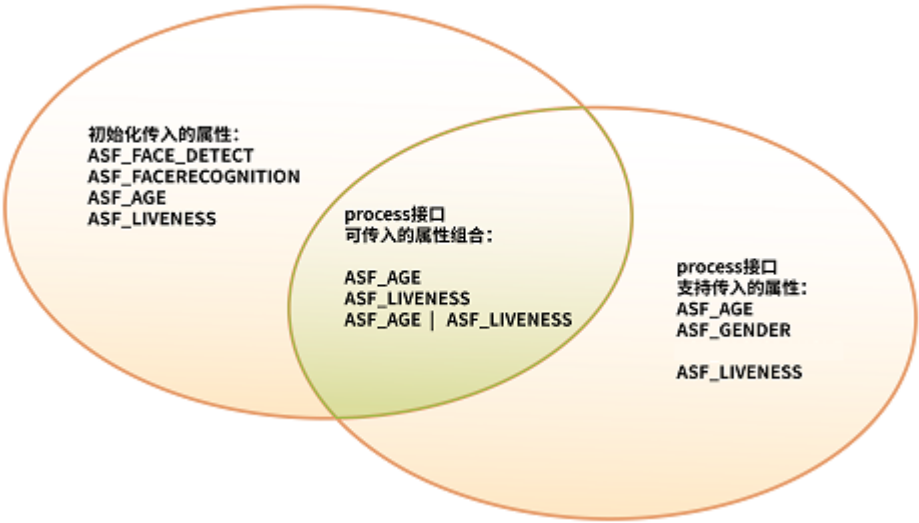
重要参数说明

- **combinedMask**

process接口中支持检测ASF_AGE、ASF_GENDER、ASF_LIVENESS三种属性，但是想检测这些属性，必须在初始化引擎接口中对想要检测的属性进行初始化。

关于初始化接口中combinedMask和process接口中combinedMask参数之间的关系，举例进行详细说明，如下图所示：

- 1. process接口中combinedMask支持传入的属性有ASF_AGE、ASF_GENDER、ASF_LIVENESS。
- 2. 初始化中传入了ASF_FACE_DETECT、ASF_FACERECOGNITION、ASF_AGE、ASF_LIVENESS属性。
- 3. process可传入属性组合只有ASF_AGE、ASF_LIVENESS、ASF_AGE | ASF_LIVENESS。



返回值

成功返回MOK，失败详见[4.2 错误码列表](#)。

示例代码

下面代码要求初始化引擎时包含了ASF_AGE | ASF_GENDER

```

MInt32 mask = ASF_AGE | ASF_GENDER;
MInt32 format = ASVL_PAF_NV12;
ASF_MultiFaceInfo faceInfo = {0};
//获取人脸位置信息
[engine detectFacesWithWidth:picwidth height:picHeight data:nv12Data format:format
faceRes:fdResult];
//进行相关信息检测
[engine processWithWidth:picwidth height:picHeight data:nv12Data format:format
faceRes:fdResult mask:mask];

```

3.5.19 getAge

功能描述

获取年龄信息。

方法

```
- (MRESULT) getAge: (LPASF_AgeInfo) ageInfo;
```

参数说明

参数	类型	描述
ageInfo	out	检测到的年龄信息数组

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

前提： `process` 接口调用成功，且 `combinedMask` 参数中包含 `ASF_AGE` 属性

```

ASF_AgeInfo ageInfo = {0};
[engine getAge:&ageInfo];

```

3.5.20 getGender

功能描述

获取性别信息。

方法

```
- (MRESULT)getGender:(LPASF_GenderInfo)genderInfo;
```

参数说明

参数	类型	描述
genderInfo	out	检测到的性别信息数组

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

前提：`process` 接口调用成功，且 `combinedMask` 参数中包含 `ASF_GENDER` 属性

```
ASF_GenderInfo genderInfo = {0};  
[engine getGender:&genderInfo];
```

3.5.21 getVersion

功能描述

获取SDK版本信息。

方法

```
- (NSString*)getVersion;
```

返回值

成功返回版本号。

示例代码

```
[engine getVersion];
```

3.5.22 setLivenessThreshold

功能描述

设置活体 检测阈值。

方法

- (MRESULT)setLivenessThreshold:(LPASF_LivenessInfo)threshold;

参数说明

参数	类型	描述
threshold	in	活体检测阈值[0,1]，默认值0.5

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

前提：`initFaceEngine` 接口调用成功，且combinedMask参数中包含 `ASF_LIVENESS` 属性

```
ASF_LivenessThreshold liveNessThread;
liveNessThread.thresholdmodel_BGR = 0.6;
mr = [self.arcsoftFace setLivenessThreshold:(&liveNessThread)];
if (mr == MOK)
{
    NSLog(@"setLivenessThreshold succeed");
} else {
    NSLog(@"setLivenessThreshold failed errcode %ld", mr);
}
```

3.5.23 getLivenessThreshold

功能描述

获取活体检测阈值。

方法

- (MRESULT)getLivenessThreshold:(LPASF_LivenessThreshold)livenessThreshold;

参数说明

参数	类型	描述
threshold	in	活体检测阈值[0,1]，默认值0.5

返回值

成功返回 `MOK`，失败详见 [4.2 错误码列表](#)。

示例代码

前提：initFaceEngine 接口调用成功，且combinedMask参数中包含 ASF_LIVENESS 属性

```
ASF_LivenessThreshold liveNessThreadParam;
mr = [self.arcsoftFace getLivenessThreshold:&liveNessThreadParam)];
if (mr == MOK)
{
    NSLog(@"getLivenessThreshold succeed, bgr:%f",
liveNessThreadParam.thresholdmodel_BGR);
} else {
    NSLog(@"getLivenessThreshold failed errcode %ld", mr);
}
```

3.5.24 getLiveness

功能描述

获取活体信息。

方法

```
- (MRESULT)getLiveness:(LPASF_LivenessInfo)livenessScore;
```

参数说明

参数	类型	描述
livenessScore	out	检测到的活体信息数组

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

前提：process 接口调用成功，且combinedMask参数中包含 ASF_LIVENESS 属性

```
ASF_LivenessInfo livenessInfo;
mr = [self.arcsoftFace getLiveness:&livenessInfo];
if (mr == MOK)
{
    NSLog(@"getLiveness succeed, num:%d gender:%d", livenessInfo.num,
    livenessInfo.isLive[0]);
} else {
    NSLog(@"getLiveness failed errcode %ld", mr);
}
```

3.5.25 unInitFaceEngine

功能描述

销毁SDK引擎。

方法

```
- (MRESULT)unInitFaceEngine;
```

返回值

成功返回 MOK，失败详见 [4.2 错误码列表](#)。

示例代码

```
[engine unInitFaceEngine];
```


4.2 错误码列表

错误码名	十六进制	十进制	描述
MOK	0x00	0	成功
MERR_BASIC_BASE	0x1	1	通用错误类型
MERR_INVALID_PARAM	0x2	2	无效的参数
MERR_UNSUPPORTED	0x3	3	引擎不支持
MERR_NO_MEMORY	0x4	4	内存不足
MERR_BAD_STATE	0x5	5	状态错误
MERR_USER_CANCEL	0x6	6	用户取消相关操作
MERR_EXPIRED	0x7	7	操作时间过期
MERR_USER_PAUSE	0x8	8	用户暂停操作
MERR_BUFFER_OVERFLOW	0x9	9	缓冲上溢
MERR_BUFFER_UNDERFLOW	0xA	10	缓冲下溢
MERR_NO_DISKSPACE	0xB	11	存贮空间不足
MERR_COMPONENT_NOT_EXIST	0xC	12	组件不存在
MERR_GLOBAL_DATA_NOT_EXIST	0xD	13	全局数据不存在
MERR_FSDK_BASE	0x7000	28672	FreeSDK通用错误类型
MERR_FSDK_INVALID_APP_ID	0x7001	28673	无效的AppId
MERR_FSDK_INVALID_SDK_ID	0x7002	28674	无效的SDKkey
MERR_FSDK_INVALID_ID_PAIR	0x7003	28675	AppId和SDKKey不匹配
MERR_FSDK_MISMATCH_ID_AND_SDK	0x7004	28676	SDKKey和使用的SDK不匹配
MERR_FSDK_SYSTEM_VERSION_UNSUPPORTED	0x7005	28677	系统版本不被当前SDK所支持
MERR_FSDK_LICENCE_EXPIRED	0x7006	28678	SDK有效期过期，需要重新下载更新
MERR_FSDK_APS_ENGINE_HANDLE	0x11001	69633	引擎句柄非法
MERR_FSDK_APS_MEARC_HANDLE	0x11002	69634	内存句柄非法
MERR_FSDK_APS_DEVICE_ID_INVALID	0x11003	69635	Device ID 非法
MERR_FSDK_APS_DEVICE_ID_UNSUPPORTED	0x11004	69636	Device ID 不支持
MERR_FSDK_APS_MODEL_HANDLE	0x11005	69637	模板数据指针非法
MERR_FSDK_APS_MODEL_SIZE	0x11006	69638	模板数据长度非法
MERR_FSDK_APS_IMAGE_HANDLE	0x11007	69639	图像结构体指针非法
MERR_FSDK_APS_IMAGE_FORMAT_UNSUPPORTED	0x11008	69640	图像格式不支持
MERR_FSDK_APS_IMAGE_PARAM	0x11009	69641	图像参数非法
MERR_FSDK_APS_IMAGE_SIZE	0x1100A	69642	图像尺寸大小超过支持范围

错误码名	十六进制	十进制	描述
MERR_FSDK_APS_DEVICE_AVX2_UNSUPPORTED	0x1100B	69643	处理器不支持AVX2指令
MERR_FSDK_FR_ERROR_BASE	0x12000	73728	FaceRecognition错误类型
MERR_FSDK_FR_INVALID_MEMORY_INFO	0x12001	73729	无效的输入内存
MERR_FSDK_FR_INVALID_IMAGE_INFO	0x12002	73730	无效的输入图像参数
MERR_FSDK_FR_INVALID_FACE_INFO	0x12003	73731	无效的脸部信息
MERR_FSDK_FR_NO_GPU_AVAILABLE	0x12004	73732	当前设备无GPU可用
MERR_FSDK_FR_MISMATCHED_FEATURE_LEVEL	0x12005	73733	待比较的两个人脸特征的版本不一致
MERR_FSDK_FACEFEATURE_ERROR_BASE	0x14000	81920	人脸特征检测错误类型
MERR_FSDK_FACEFEATURE_UNKNOWN	0x14001	81921	人脸特征检测错误未知
MERR_FSDK_FACEFEATURE_MEMORY	0x14002	81922	人脸特征检测内存错误
MERR_FSDK_FACEFEATURE_INVALID_FORMAT	0x14003	81923	人脸特征检测格式错误
MERR_FSDK_FACEFEATURE_INVALID_PARAM	0x14004	81924	人脸特征检测参数错误
MERR_FSDK_FACEFEATURE_LOW_CONFIDENCE_LEVEL	0x14005	81925	人脸特征检测结果置信度低
MERR_FSDK_FACEFEATURE_EXPIRED	0x14006	81926	人脸特征检测结果操作过期
MERR_FSDK_FACEFEATURE_MISSFACE	0x14007	81927	人脸特征检测人脸丢失人脸特征检测结果置信度低
MERR_FSDK_FACEFEATURE_NO_FACE	0x14008	81928	人脸特征检测没有人脸
MERR_FSDK_FACEFEATURE_FACEDATA	0x14009	81929	人脸特征检测人脸信息错误
MERR_FSDK_FACE_FEATURE_STATUES_ERROR	0x1400A	81930	人脸特征检测人脸状态错误
MERR_ASF_EX_BASE	0x15000	86016	ASF错误类型
MERR_ASF_EX_FEATURE_UNSUPPORTED_ON_INIT	0x15001	86017	Engine不支持的检测属性
MERR_ASF_EX_FEATURE_UNINITED	0x15002	86018	需要检测的属性未初始化
MERR_ASF_EX_FEATURE_UNPROCESSED	0x15003	86019	待获取的属性未在process中处理过
MERR_ASF_EX_FEATURE_UNSUPPORTED_ON_PROCESS	0x15004	86020	PROCESS不支持的检测属性组合，例如FR，有自己独立的处理函数
MERR_ASF_EX_INVALID_IMAGE_INFO	0x15005	86021	无效的输入图像
MERR_ASF_EX_INVALID_FACE_INFO	0x15006	86022	无效的脸部信息
MERR_ASF_ACTIVE_BASE	0x16000	90112	激活结果类型
MERR_ASF_ACTIVATION_FAIL	0x16001	90113	SDK激活失败,请打开读写权限
MERR_ASF_ALREADY_ACTIVATED	0x16002	90114	SDK已激活
MERR_ASF_NOT_ACTIVATED	0x16003	90115	SDK未激活
MERR_ASF_SCALE_NOT_SUPPORT	0x16004	90116	detectFaceScaleVal不支持
MERR_ASF_ACTIVEFILE_SDKTYPE_MISMATCH	0x16005	90117	激活文件与SDK类型不匹配，请确认使用的sdk
MERR_ASF_DEVICE_MISMATCH	0x16006	90118	设备不匹配
MERR_ASF_UNIQUE_IDENTIFIER_ILLEGAL	0x16007	90119	唯一标识不合法

错误码名	十六进制	十进制	描述
MERR_ASF_PARAM_NULL	0x16008	90120	参数为空
MERR_ASF_LIVENESS_EXPIRED	0x16009	90121	活体已过期
MERR_ASF_VERSION_NOT_SUPPORT	0x1600A	90122	版本不支持
MERR_ASF_SIGN_ERROR	0x1600B	90123	签名错误
MERR_ASF_DATABASE_ERROR	0x1600C	90124	激活信息保存异常
MERR_ASF_UNIQUE_CHECKOUT_FAIL	0x1600D	90125	唯一标识符校验失败
MERR_ASF_COLOR_SPACE_NOT_SUPPORT	0x1600E	90126	颜色空间不支持
MERR_ASF_IMAGE_WIDTH_HEIGHT_NOT_SUPPORT	0x1600F	90127	图片宽高不支持，宽度需四字节对齐
MERR_ASF_ACTIVATION_DATA_DESTROYED	0x16011	90129	激活数据被破坏,请删除激活文件，重新进行激活
MERR_ASF_SERVER_UNKNOWN_ERROR	0x16012	90130	服务端未知错误
MERR_ASF_INTERNET_DENIED	0x16013	90131	INTERNET权限被拒绝
MERR_ASF_ACTIVEFILE_SDK_MISMATCH	0x16014	90132	激活文件与SDK版本不匹配,请重新激活
MERR_ASF_DEVICEINFO_LESS	0x16015	90133	设备信息太少，不足以生成设备指纹
MERR_ASF_LOCAL_TIME_NOT_CALIBRATED	0x16016	90134	客户端时间与服务器时间（即北京时间）前后相差在30分钟以上
MERR_ASF_APPID_DATA_DECRYPT	0x16017	90135	数据校验异常
MERR_ASF_APPID_APPKEY_SDK_MISMATCH	0x16018	90136	传入的APP_ID和AppKey与使用的SDK版本不一致
MERR_ASF_NO_REQUEST	0x16019	90137	短时间大量请求会被禁止请求,30分钟之后解封
MERR_ASF_ACTIVE_FILE_NO_EXIST	0x1601A	90138	激活文件不存在
MERR_ASF_DETECT_MODEL_UNSUPPORTED	0x1601B	90139	检测模型不支持，请查看对应接口说明，使用当前支持的检测模型
MERR_ASF_CURRENT_DEVICE_TIME_INCORRECT	0x1601C	90140	当前设备时间不正确，请调整设备时间
MERR_ASF_ACTIVATION_QUANTITY_OUT_OF_LIMIT	0x1601D	90141	年度激活数量超出限制，次年清零
MERR_ASF_NETWORK_BASE	0x17000	94208	网络错误类型
MERR_ASF_NETWORK_COULDNT_RESOLVE_HOST	0x17001	94209	无法解析主机地址
MERR_ASF_NETWORK_COULDNT_CONNECT_SERVER	0x17002	94210	无法连接服务器
MERR_ASF_NETWORK_CONNECT_TIMEOUT	0x17003	94211	网络连接超时
MERR_ASF_NETWORK_UNKNOWN_ERROR	0x17004	94212	网络未知错误
MERR_ASF_ACTIVEKEY_COULDNT_CONNECT_SERVER	0x18001	98305	无法连接激活服务器
MERR_ASF_ACTIVEKEY_SERVER_SYSTEM_ERROR	0x18002	98306	服务器系统错误
MERR_ASF_ACTIVEKEY_POST_PARM_ERROR	0x18003	98307	请求参数错误
MERR_ASF_ACTIVEKEY_SDK_FILE_MISMATCH	0x18008	98312	SDK与激活文件版本不匹配
MERR_ASF_ACTIVEKEY_DEVICE_OUT_OF_LIMIT	0x1800A	98314	批量授权激活码设备数超过限制
MERR_ASF_ACTIVE_EX_BASE	0x20000	131072	激活结果类型

错误码名	十六进制	十进制	描述
MERR_ASF_ACTIVE_FILE_NO_EXIST	0x20001	131073	激活文件不存在
MERR_ASF_PACKAGEID_MISMATCH	0x20002	131074	包名不匹配
MERR_ASF_PACKAGE_SIGN_MISMATCH	0x20003	131075	包签名不匹配
MERR_ASF_DEVICE_MISMATCH	0x20004	131076	设备不匹配，请重新激活
MERR_ASF_CURRENT_DEVICE_TIME_INCORRECT	0x20005	131077	当前设备时间不正确，请调整设备时间
MERR_ASF_ACTIVATION_DATA_DESTROYED	0x20006	131078	激活数据被破坏,请删除激活文件，重新进行激活
MERR_ASF_ACTIVEFILE_SDK_MISMATCH	0x20007	131079	激活文件与SDK版本不匹配,请重新激活
MERR_ASF_SDK_SIGN_CHECK_ERROR	0x20008	131080	SDK签名校验错误
MERR_ASF_INIT_BASE	0x21000	135168	初始化错误
MERR_ASF_AUTHORIZATION_EXPIRED	0x21002	135170	包授权已过期
MERR_ASF_FILE_SDK_INFO_MISMATCH	0x21004	135172	激活文件与SDK基础信息不匹配
MERR_ASF_SERVER_BASE	0x23000	143360	网络错误类型
MERR_ASF_PARAM_FORMAT_ERROR	0x23001	143361	请求数据格式错误
MERR_ASF_APPID_SDKKEY_PACKAGEID_ERROR	0x23003	143363	APPID SDKKEY PACKAGEID存在格式错误
MERR_ASF_APPID_PACKAGEID_MISMATCH	0x23004	143364	APPID、PACKAGEID组合不存在
MERR_ASF_APPID_PACKAGEID_EXPIRED	0x23005	143365	APPID、PACKAGEID组合授权已过期
MERR_ASF_SERVER_PACKAGE_SIGN_MISMATCH	0x23006	143366	包签名验证不通过
MERR_ASF_ILLEGAL_REQUEST	0x23007	143367	非法请求
MERR_ASF_SERVER_SIGN_CHECK_ERROR	0x23008	143368	服务端签名校验错误
MERR_ASF_SERVER_DECRYPTION_FAILED	0x23009	143369	服务端解密失败
MERR_ASF_SHORT_TIME_LARGE_REQUEST	0x2300A	143370	短时间大量请求会被禁止请求,30分钟之后解封
MERR_ASF_ACTIVE_LIMIT_REACHED	0x2300D	143373	有效期内授权次数已超出
MERR_ASF_SEARCH_EMPTY	0x25001	151553	人脸列表为空
MERR_ASF_SEARCH_NO_EXIST	0x25002	151554	人脸不存在
MERR_ASF_SEARCH_FEATURE_SIZE_MISMATCH	0x25003	151555	特征值长度不匹配
MERR_ASF_SEARCH_LOW_CONFIDENCE	0x25004	151556	相似度异常
MERR_ASF_SEARCH_NUM_OVER_LIMIT	0x25005	151557	查询人脸的数量超过当前注册人脸数量
MERR_ASF_UNRESTRICTED_TIME_LIMIT_EXCEEDED	0x30000	196608	非限制时段数量超出(限制时段：周一到周五 9：00-14：00)

4.3 FAQ

Q：如何将人脸识别1:1进行开发改为1:n？

A：先将人脸特征数据用本地文件、数据库或者其他的方式存储下来，若检测出结果需要显示图像可以保存对应的图像。之后循环对特征值进行对比，相似度最高者若超过您设置的阈值则输出相关信息。

Q：初始化引擎时（detectFaceScaleVal）参数多大比较合适？

A：用于数值化表示的最小人脸尺寸，该尺寸代表人脸尺寸相对于图片长边的占比。VIDEO 模式有效值范围[2,32]，推荐值为16；IMAGE 模式有效值范围[2,32]，推荐值为30，特殊情况下可根据具体场景进行设置。

Q：初始化引擎之后调用其他接口返回错误码86018，该怎么解决？

A：86018即需要检测的属性未初始化，需要查看调用接口的属性值有没有在初始化引擎时在combinedMask参数中加入。

Q：调用detectFaces、extractFaceFeature和process接口返回90127错误码，该怎么解决？

A：ArcFace SDK对图像尺寸做了限制，宽高大于0，宽度为4的倍数，NV12格式的图片高度为2的倍数，BGR24格式的图片高度不限制；如果遇到90127请检查传入的图片尺寸是否符合要求，若不符合可对图片进行适当的裁剪。

Q：人脸检测结果的人脸框Rect为何有时会溢出传入图像的边界？

A：Rect溢出边界可能是人脸只有一部分在图像中，算法会对人脸的位置进行估计。

Q：SDK是否支持多线程调用？

A：SDK支持多线程调用，但是在不同线程使用同一算法功能时需要使用不同引擎对象，且调用过程中不能销毁。

Q：哪些因素会影响人脸检测、人脸跟踪、人脸特征提取等SDK调用所用时间？

A：硬件性能、图片质量等。